ตารางสรุปผล

	Workload / Policy	Workers	Printer	DB	avg WT	avg TAT	Throughput	avg RW
1	standard / FCFS	3	1	2	2970 ms	5478 ms	1.14 jobs/sec	76 ms
2	standard / Priority	3	1	2	2339 ms	4972 ms	1.07 jobs/sec	252 ms
3	standard / Priority	1	1	2	8539 ms	10991 ms	0.41 jobs/sec	1 ms
4	standard / Priority	5	1	2	1135 ms	4091 ms	1.29 jobs/sec	716 ms
5	printer / Priority	3	1	2	1163 ms	4279 ms	0.79 jobs/sec	1625 ms
6	printer / Priority	3	2	2	779 ms	2572 ms	1.41 jobs/sec	303 ms
7	standard / Priority (repeat)	3	1	2	2331 ms	4958 ms	1.07 jobs/sec	256 ms

1. เพิ่ม Worker จาก 3 เป็น 5 แล้ว Throughput เพิ่มตามสัดส่วนหรือไม่ และ avg Resource Wait เปลี่ยนอย่างไร เพราะอะไร?

- Throughput ไม่ได้เพิ่มขึ้นตามสัดส่วน
    - จากตาราง การเพิ่ม Worker จาก 3 เป็น 5 ตัว (เพิ่มขึ้น 66.7%) ส่งผลให้ Throughput เพิ่มขึ้นเพียงเล็กน้อยจาก 1.07 เป็น 1.29 jobs/sec (เพิ่มขึ้นเพียง 20.5%)
    - สาเหตุ: เกิดคอขวดที่ทรัพยากรร่วม (Resource Bottleneck) แม้จะมี Worker (CPU) เพิ่มขึ้นมารับงานได้เร็วขึ้น แต่โควตาของ Printer (1 เครื่อง) และ Database (2 เครื่อง) เท่าเดิม ทำให้งานไปติดกระจุกสะสมในขั้นตอนขอใช้ทรัพยากร
- avg Resource Wait (avg RW) พุ่งสูงขึ้นอย่างเห็นได้ชัด
    - ค่า avg RW เพิ่มขึ้นจาก 252 ms เป็น 716 ms (เพิ่มขึ้นเกือบ 3 เท่า)
    - สาเหตุ: เกิดการแย่งชิงทรัพยากร (Resource Contention) เมื่อมี Worker รันงานพร้อมกัน 5 ตัว ทำให้มี Worker ที่ประมวลผล CPU เสร็จในเวลาใกล้เคียงกัน แล้วเข้ามาแย่งกันจอง Semaphore ของ Printer/Database มากขึ้น แถวคิวรอใน ResourceManager จึงยาวขึ้น

2. FCFS กับ Priority ทำให้ Job กลุ่มใดได้ประโยชน์หรือเสียประโยชน์จาก jobs_standard.csv? (อ้างอิง Waiting Time และลำดับ START ใน log) และเหตุใด Priority ถึงอาจมี Throughput ต่ำกว่า FCFS แม้ค่าเฉลี่ย Waiting Time จะดีกว่า?

- กลุ่มงานที่ได้ประโยชน์: งานความสำคัญสูง (Priority 1-2 เช่น J07, J08)
    - แม้จะเดินทางมาถึงระบบทีหลัง (1006 ms) แต่ได้แซงคิวขึ้นมา START ทำงานทันทีเมื่อ Worker ว่าง (2552 ms)
    - ส่งผลให้เวลารอคิวใน ReadyQueue ภาพรวม (avg WT) ลดลงจาก 2970 ms เหลือ 2339 ms
- กลุ่มงานที่เสียประโยชน์: งานความสำคัญต่ำ (Priority 5 เช่น J04, J05)
    - แม้จะเดินทางมาถึงระบบตั้งแต่ช่วงแรก (300 - 400 ms) แต่ถูกเบียดคิวไป START ช้ามาก (5334 - 5542 ms) เกิดภาวะ Starvation ชั่วคราว
    - งานกลุ่มนี้จะได้ประโยชน์มากกว่าหากใช้นโยบาย FCFS
- เหตุผลที่ Priority มี Throughput ต่ำกว่า FCFS แม้ avg WT จะดีกว่า:
    - FCFS (avg RW = 76 ms): กระจายการใช้งาน Printer และ Database ออกไปตามลำดับเวลาที่งานมาถึง งานจึงทยอยเคลียร์ออกจากระบบได้สม่ำเสมอ
    - Priority (avg RW = 252 ms): เร่งงานความสำคัญสูง (J07, J08, J09) ขึ้นมาทำพร้อมกัน แต่รูปแบบงานกลุ่มนี้มีความต้องการใช้ PRINTER และ DATABASE เหมือนกันพอดี
    - ผลกระทบรวม: แม้ค่า avg WT ใน ReadyQueue จะดีขึ้นเพราะงานสำคัญหลุดจากคิว CPU ได้เร็ว แต่ Worker กลับต้องไปติดแหง็กต่อคิวรอ Semaphore ใน ResourceManager แทน ส่งผลให้อัตราการเคลียร์งานสำเร็จต่อวินาที (Throughput) ลดลงจาก 1.14 เหลือ 1.07 jobs/sec

3. เมื่อเพิ่ม Printer permit จาก 1 เป็น 2 แล้ว avg Resource Wait และ Throughput เปลี่ยนอย่างไร และจุดคอขวดของระบบย้ายไปอยู่ที่ใด?

- การเปลี่ยนแปลงของ avg Resource Wait (avg RW) และ Throughput:
    - avg Resource Wait (avg RW) ลดลงอย่างมาก จาก 1625 ms เหลือเพียง 303 ms
    - Throughput เพิ่มขึ้นเกือบเท่าตัว จาก 0.79 เป็น 1.41 jobs/sec
- การย้ายของจุดคอขวดระบบ:
    - จุดคอขวดเดิม: อยู่ที่ Printer เนื่องจากมี Permit เพียง 1 เครื่อง แต่งานเน้นใช้ Printer จำนวนมาก ทำให้เกิดคอขวดอย่างรุนแรง
    - จุดคอขวดใหม่: ย้ายกลับไปอยู่ที่ จำนวน Worker (CPU) และระยะเวลาการประมวลผลหลัก (workMs) เมื่อเพิ่ม Printer เป็น 2 เครื่อง ทำให้ไม่มี Worker ตัวใดต้องนั่งรอคิวเครื่องพิมพ์นานๆ ระบบจึงประมวลผลได้เต็มกำลังตามจำนวน Worker ที่มี

4. เปรียบเทียบแถว 2 กับแถว 7: เหตุใด workload และชุดค่าที่ใช้รันเหมือนเดิม จึงอาจให้เวลาใน log ลำดับเหตุการณ์ และค่าที่วัดต่างกันเล็กน้อย?

- เกิดจากธรรมชาติของระบบ Multithreading (Non-deterministic Execution)
    - Thread Scheduling ของ OS: ตัวจัดการ Thread ของระบบปฏิบัติการ (OS Thread Scheduler) จะจัดสรรเวลา CPU (Time Slices) ให้กับแต่ละ Thread ในแต่ละรอบการรันไม่เหมือนกันเป๊ะๆ
    - ความคลาดเคลื่อนของ เวลา (Time Jitter): คำสั่ง Thread.sleep() ใน Java ไม่ได้การันตีตื่นตรงเวลาในระดับ millisecond เป๊ะๆ ขึ้นอยู่กับภาระงานของเครื่องขณะนั้น
    - การแข่งขันจอง Lock / Semaphore: จังหวะที่ Worker แต่ละตัวยื่นเรื่องขอจอง Semaphore หรือเข้าดึงงานจาก ReadyQueue อาจเหลื่อมล้ำกันเล็กน้อยในระดับมิลลิวินาที ทำให้ลำดับ Log หรือค่าเฉลี่ยสถิติต่างกันเล็กน้อย (เช่น avg WT 2339 ms vs 2331 ms) ซึ่งถือเป็นเรื่องปกติของระบบขนาน